

```

57.
58. # Sort by Sharpe Ratio (or another preferred metric)
59. results_df = results_df.sort_values('Sharpe Ratio', ascending=False)
60.
61. return results_df
62.
63. Example usage:
64. # Define parameter space
65. param_space = {
66.     'short_ma': (5, 50), # Min and max values for short MA
67.     'long_ma': (50, 200) # Min and max values for long MA
68. }
69.
70. # Perform random search
71. results = random_search_optimization('AAPL', '2018-01-01', '2023-01-01', param_space,
n_iterations=100)
72.
73. # Display top 5 parameter combinations
74. print(results.head(5))
75.

```

Random search tends to find good (though perhaps not optimal) parameters more efficiently than grid search, especially in high-dimensional parameter spaces.

Genetic Algorithms: Evolution-Inspired Optimization

Genetic algorithms take inspiration from natural selection to efficiently search for optimal parameters. They start with a population of random parameter sets, evaluate their performance, and then create a new generation by combining and mutating the best-performing sets—mimicking natural selection and evolution.

While a full implementation of genetic algorithms is beyond our scope, here's a simplified example:

```

1. def genetic_algorithm_optimization(ticker, start_date, end_date, param_bounds,
2.                                     population_size=50, generations=10):
3.     """
4.     Perform genetic algorithm optimization for a moving average crossover strategy

```